Heap

You are given an unsorted array of n integers and an integer k.

Write a program to find the k-th smallest element in the array using a heap-based approach.

Solution (Heap):

Instead of sorting the entire array, use a **max heap of size k**.

Process:

- Insert the first K elements into a max heap.
- Traverse the remaining elements.
- If the current element is smaller than the heap's top (largest of the k smallest so far), remove the top and insert the new element.
- After processing all elements, the heap top is the **k-th smallest element**.

Time Complexity:
O(n log k)

For our problem
k = 3
(b/c homeworks)

Example: 7, 10, 4, 3, 20, 15 n items → list of n items
Find 3 smallest items k=3

Find Final Answer: 3, 4, 7, 10, 15, 20 → list of k items

Actual answer = 7 because
7 is the 3rd smallest item

Strategy: Create max heap to store 3 (k^{th}) smallest items

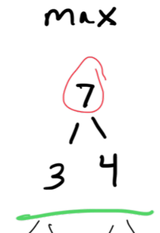
> Why max heap? 3, 4, 7

Min heap stores smallest item at root

min

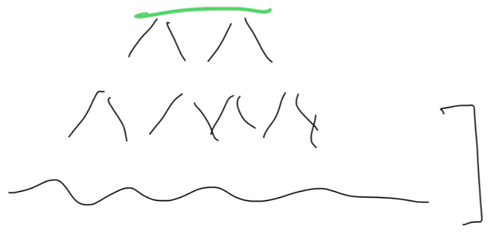
vs

max



Max heap stores largest item at root

50% of the items are in the leaves so inefficient to search max item here

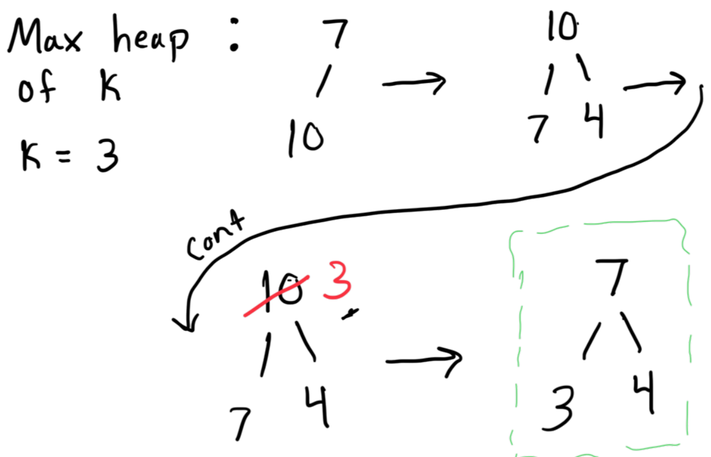


50% of items are in leaves so inefficient to search min item here

We need our heap to be size k
If k=3 then heap size = 3

Back to example: 7, 10, 4, 3, 20, 15

Max heap of K, K = 3:



1. Compare 3 to root
   3 < 10
2. Replace 10 with 3
3. Bubble down

7 < 20 ✓
7 < 15 ✓

Final heap

To get 3rd smallest, get the root (.front()?)

Time complexity / Runtime?

Heap insert / remove → O(log k)
# of elements → we do go through n items but they don't all go in heap

Think of fast find/exponential search

Total runtime: n * log k = **O(n log k)**